

GPUMODE Lecture Study Notes: Lecture 35

SGLang

Core Problem the Lecture Addresses

This lecture studies **SGLang** as an open-source large language model inference engine and focuses on three system-level optimizations that matter when serving autoregressive models at low latency and high throughput:

1. **CPU and GPU overlap:** reduce scheduler overhead so that the GPU does not sit idle while Python-side scheduling logic prepares the next decoding batch.
2. **FlashInfer integration:** use specialized GPU kernels for attention, sampling, normalization, activation, paged key-value cache access, and better work scheduling under variable sequence lengths.
3. **TurboMind GEMM optimization:** accelerate quantized linear layers, especially mixed-precision GEMM such as FP16 activations multiplied by INT4 weights.

The central systems problem is that LLM inference is not only a kernel-performance problem. A serving engine must continuously batch requests, reuse prefix key-value cache, allocate memory, check stopping criteria, decode tokens, and stream outputs. Even if individual CUDA kernels are fast, the GPU can be underutilized if the CPU scheduler becomes the bottleneck.

A useful mental model is: an inference engine is a pipeline, not a single CUDA kernel. The GPU is the expensive stage, but the CPU scheduler controls whether that expensive stage is continuously fed with work.

Required Background

Autoregressive LLM Inference

For an autoregressive transformer, generation proceeds token by token. Given a prompt token sequence

$$x_{1:L} = (x_1, x_2, \dots, x_L),$$

the model computes a distribution over the next token:

$$p(x_{L+1} \mid x_{1:L}).$$

After sampling or selecting a token, the sequence becomes

$$x_{1:L+1} = (x_1, x_2, \dots, x_L, x_{L+1}),$$

and the process repeats until an end-of-sequence token, a stop string, or a length limit is reached.

The important serving consequence is that each request repeatedly alternates between GPU model execution and CPU-side request management. The per-token loop is short, so even small CPU overheads can become visible.

Prefill and Decode

LLM serving is commonly split into two phases:

- **Prefill:** process the full prompt and build the key-value cache for all prompt tokens. This phase usually has higher arithmetic intensity because many query tokens attend over many key-value tokens.
- **Decode:** generate one token per active request per step. This phase is often memory-bandwidth sensitive because each new query attends to the existing key-value cache.

If a batch contains B active sequences and sequence i has current length L_i , then a decode step reads key-value cache proportional to

$$\sum_{i=1}^B L_i.$$

When sequence lengths vary strongly, work imbalance appears: some sequences require much more key-value cache traffic than others.

Continuous Batching

In offline batching, a fixed batch of requests is processed together. In online serving, requests arrive and finish at different times. **Continuous batching** allows an engine to add new requests and remove finished requests at every decode step.

At step t , the engine maintains an active batch

$$\mathcal{B}_t = \{r_1, r_2, \dots, r_{B_t}\},$$

where each request r_i has its own sequence length, sampling parameters, stopping criteria, and key-value cache allocation. The scheduler chooses the next runnable batch:

$$\mathcal{B}_{t+1} = \text{Schedule}(\mathcal{Q}_t, \mathcal{B}_t, \mathcal{M}_t),$$

where $\mathcal{Q}_t$ is the waiting queue and $\mathcal{M}_t$ is the memory state.

Key–Value Cache

For each transformer layer ℓ , attention stores keys and values:

$$K^{(\ell)} \in \mathbb{R}^{B \times H_{kv} \times L \times d_h}, \quad V^{(\ell)} \in \mathbb{R}^{B \times H_{kv} \times L \times d_h},$$

where H_{kv} is the number of key–value heads and d_h is the head dimension. The cache avoids recomputing keys and values for old tokens.

The approximate key–value cache size per request is

$$\text{Bytes}_{KV} = 2 \cdot N_{layers} \cdot H_{kv} \cdot L \cdot d_h \cdot s,$$

where the factor 2 accounts for keys and values, and s is the number of bytes per scalar.

Time to First Token and Inter-Token Latency

Two serving metrics appear throughout the lecture:

- **TTFT**: time to first token. This includes request admission, prefill, scheduling, and producing the first generated token.
- **ITL**: inter-token latency. This measures the latency between consecutive generated tokens during decode.

A serving engine often optimizes a throughput-latency frontier. Higher request rate can improve GPU utilization, but it can also increase TTFT and ITL because requests wait longer in queues.

Main Concepts

SGLang as an Inference Engine

The lecture presents SGLang as a lightweight but high-performance inference engine. The important design goal is not merely to expose an API for LLM serving, but to provide an engine that is customizable enough for systems researchers and production teams to modify.

The transcript highlights that SGLang evolved from a programming paradigm for language model applications into a full inference engine. Important milestones mentioned include prefix caching, constrained decoding, performance-focused releases, FlashInfer integration, and support for specialized attention or model features.

The Three Optimization Themes

The three main technical themes can be summarized as:

1. **Scheduling pipeline**: make CPU scheduling nonblocking relative to GPU execution.

2. **Attention backend:** use FlashInfer for high-performance attention and related kernels, including paged key-value cache and Stream-K-like scheduling ideas.
3. **Quantized GEMM backend:** integrate high-performance low-bit matrix multiplication from TurboMind to reduce reliance on external components for linear layers.

These are complementary. CPU overlap improves utilization at the engine level. FlashInfer improves transformer-kernel efficiency. TurboMind improves matrix multiplication, especially for quantized weights.

Why CPU Overhead Matters

The lecture emphasizes that a serving engine can waste a large fraction of time in CPU scheduling. This is especially damaging when each GPU step is short, as in decode with small or moderate batch sizes. The CPU must:

- receive new user input messages;
- process model outputs from the previous batch;
- check stop conditions such as end-of-sequence tokens and length limits;
- perform prefix matching and key-value cache reuse decisions;
- reorder requests for batching efficiency;
- allocate memory for the next batch;
- launch GPU work and stream outputs back to users.

If these operations are performed in a blocking loop, the timeline looks like:

CPU schedule $\rightarrow$ GPU execute $\rightarrow$ CPU process $\rightarrow$ GPU execute $\rightarrow \dots$

The GPU is idle during CPU scheduling. The CPU is idle or blocked while waiting for GPU completion. The desired timeline is:

CPU schedule batch $t + 1$ $\parallel$ GPU execute batch t .

Blocking Scheduler Loop

A simplified blocking scheduler can be written as follows.

```
def blocking_scheduler_loop():
    while True:
        recv_new_requests()
        process_finished_outputs()

        batch = get_next_batch_to_run()
        allocate_kv_cache_and_buffers(batch)
```

```

# Blocking call: CPU waits until GPU work is done.
output_tokens = model_worker.run_batch(batch)

check_stop_conditions(output_tokens)
stream_outputs_to_users(output_tokens)

```

The key problem is that `run_batch` is blocking. The scheduler cannot prepare the next batch until the current GPU batch completes. This serializes CPU and GPU work.

Overlapped Scheduler Loop

SGLang’s overlap approach makes the GPU batch execution nonblocking. The scheduler receives a future-like object immediately, queues it, and then processes outputs from earlier batches.

```

def overlapped_scheduler_loop():
    pending = Queue()

    while True:
        recv_new_requests()

        batch = get_next_batch_to_run()
        allocate_kv_cache_and_buffers(batch)

        # Nonblocking: returns a future token object.
        future_tokens = model_worker.run_batch_nonblocking(batch)
        pending.put((batch, future_tokens))

        # Process an older completed batch while the GPU works.
        old_batch, old_future = pending.get_ready_or_oldest()
        output_tokens = old_future.get()

        check_stop_conditions(output_tokens)
        stream_outputs_to_users(output_tokens)

```

The scheduler still performs almost the same logical operations, but the ordering changes. Instead of waiting immediately after launch, it uses the time during GPU execution to do useful scheduling work.

Dependency Challenge: Stop Conditions

The first major challenge is that scheduling the next decode batch depends on knowing which requests have finished. A request should be removed if its latest token is an end-of-sequence token or if it reaches a maximum length:

$$\text{finished}(r_i) = \left[x_i^{(t)} = x_{EOS} \right] \vee \left[L_i^{(t)} \geq L_{max,i} \right].$$

In a blocking loop, this condition is known before constructing the next batch. In an overlapped loop, the token value may not yet be available because GPU execution is still in flight.

SGLang’s practical solution is to **delay the finished-condition check**. The scheduler temporarily assumes the request is not finished and may decode one extra token. This trades a small amount of redundant GPU work for overlap:

$$\text{Overhead}_{extra} \approx \frac{N_{finished}}{B} \cdot C_{decode},$$

where $N_{finished}$ is the number of requests that actually finished at the previous step, B is the batch size, and C_{decode} is the cost of one decode token. When B is large and only a small fraction of requests finish at each step, the overhead is small.

Code-Reuse Challenge: Future Tokens

The second challenge is maintaining both overlap and non-overlap implementations without duplicating the scheduler. SGLang introduces a future-token abstraction. The object exposes metadata such as tensor shape immediately, while delaying access to actual token values.

A sketch of the abstraction is:

```
class FutureTokens:
    def __init__(self, shape, event, device_buffer):
        self.shape = shape
        self.event = event
        self.device_buffer = device_buffer

    def get(self):
        self.event.synchronize()
        return copy_or_view_tokens(self.device_buffer)
```

Most scheduling logic depends on shapes and request metadata, not the actual token values. Therefore the same code path can run for both blocking tensors and future tensors until a value-dependent operation is required.

Python GIL Challenge

The third challenge is Python’s Global Interpreter Lock. Python threads cannot freely execute CPU-bound Python bytecode in parallel. The lecture mentions several possible directions:

- use Python versions that support running without the GIL;
- use multiple processes instead of threads;
- make serialization between processes extremely fast;

- move selected frontend or server components into Rust;
- use freed CPU cores after CUDA graph launch for useful work.

The systems lesson is that nonblocking GPU launches are necessary but not always sufficient. True CPU overlap also depends on how much host-side logic can run concurrently without Python runtime bottlenecks.

Hardware-Level Explanation

GPU Idle Time as a Pipeline Bubble

Let T_{cpu} be the time spent scheduling and processing one decode step on the CPU, and let T_{gpu} be the time spent executing the model worker on the GPU. In a blocking design, the step time is roughly

$$T_{blocking} = T_{cpu} + T_{gpu}.$$

In an ideal overlapped design, the steady-state step time becomes

$$T_{overlap} = \max(T_{cpu}, T_{gpu}).$$

The speedup is

$$S = \frac{T_{cpu} + T_{gpu}}{\max(T_{cpu}, T_{gpu})}.$$

If CPU and GPU time are equal, the theoretical pipeline speedup approaches 2. If one side dominates, overlap helps less but still removes pipeline bubbles.

CUDA Streams, Events, and Futures

A nonblocking GPU launch usually relies on CUDA stream semantics. A kernel launch enqueues work on a stream and returns control to the host before the kernel completes. Correctness is enforced with events or stream dependencies.

A future token object is conceptually tied to a CUDA event:

$$\text{future is ready} \iff \text{CUDA event has completed}.$$

The scheduler can manipulate metadata while the device computes, but it must synchronize before accessing actual token values.

Memory Hierarchy in Decode

Decode attention repeatedly reads old key-value cache from GPU memory. The memory path is:

$$\text{HBM} \rightarrow \text{L2 cache} \rightarrow \text{SM} \rightarrow \text{registers and shared memory}.$$

For one request and one layer, the approximate bytes read for attention during one decode token are

$$\text{Bytes}_{\text{decode,layer}} \approx 2 \cdot H_{kv} \cdot L \cdot d_h \cdot s.$$

The factor 2 accounts for keys and values. Long contexts increase memory traffic linearly. This is why prefix caching, paged key–value cache layout, attention scheduling, and long-context optimization are central to serving performance.

Paged Key–Value Cache and Radix Attention

Many engines use paged key–value caches, where tokens are stored in blocks. If block size is P , a sequence of length L requires

$$N_{\text{blocks}} = \left\lceil \frac{L}{P} \right\rceil$$

blocks. Larger blocks improve contiguous access but can waste memory and reduce fine-grained prefix sharing.

The lecture discusses SGLang’s radix attention approach as using very fine-grained token-level cache matching, conceptually similar to a block size of one. This is useful for prefix sharing because two requests can reuse any shared prefix at token granularity.

The trade-off is:

- small blocks or token-level units improve prefix reuse and reduce recomputation;
- larger blocks may simplify memory management and improve locality;
- if batch size is one and no prefix sharing exists, the difference is less important;
- under multi-request serving with repeated prefixes, fine-grained reuse can improve TTFT and sometimes ITL.

Visual Concept

Draw a tree of token prefixes where each edge is a token. Show two requests sharing the same first several tokens, then branching. Annotate the shared path as reusable key–value cache and the branches as newly computed tokens.

Mathematical Reasoning

Throughput Under Blocking and Overlap

Assume each decode step emits B tokens across the active batch. Blocking throughput is

$$\Theta_{\text{blocking}} = \frac{B}{T_{\text{cpu}} + T_{\text{gpu}}}.$$

Ideal overlapped throughput is

$$\Theta_{overlap} = \frac{B}{\max(T_{cpu}, T_{gpu})}.$$

The throughput gain is

$$G = \frac{\Theta_{overlap}}{\Theta_{blocking}} = \frac{T_{cpu} + T_{gpu}}{\max(T_{cpu}, T_{gpu})}.$$

This formula explains why overlap is most valuable when CPU scheduling time is nontrivial relative to GPU execution time.

Cost of Delayed Stop Checking

If a fraction p of requests finish at a given decode step, delayed stop checking may generate one extra useless token for those requests. The expected extra compute per step is roughly

$$C_{extra} = pB \cdot C_{token},$$

where C_{token} is the marginal cost of decoding one request for one token. Relative to the normal batch cost BC_{token} , the overhead ratio is

$$\rho = \frac{pBC_{token}}{BC_{token}} = p.$$

If only a small fraction of requests finish on each step, the extra-token overhead is small compared with the utilization benefit from removing GPU idle time.

TTFT and ITL Decomposition

A simple decomposition for time to first token is

$$T_{TTFT} = T_{queue} + T_{schedule} + T_{prefill} + T_{first\ decode} + T_{sample} + T_{stream}.$$

A simple decomposition for inter-token latency is

$$T_{ITL} = T_{schedule} + T_{decode} + T_{sample} + T_{stream}.$$

CPU overlap primarily attacks $T_{schedule}$ as visible latency. FlashInfer primarily attacks $T_{prefill}$, T_{decode} , T_{sample} , and related GPU-kernel costs. TurboMind attacks linear-layer execution inside $T_{prefill}$ and T_{decode} .

Arithmetic Intensity

Arithmetic intensity is

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

The roofline model gives an upper bound:

$$\text{Throughput} \leq \min(P_{peak}, B_{mem} \cdot I),$$

where P_{peak} is peak compute throughput and B_{mem} is memory bandwidth.

Decode attention often has low arithmetic intensity because it reads large key-value cache vectors to compute a small amount of work per generated token. Quantized GEMM improves effective arithmetic intensity by reducing bytes moved for weights.

Effective Bits for INT4 Group Quantization

The lecture discusses an example where FP16 activations are multiplied by INT4 weights with group size 128. Each group stores 128 four-bit weights plus metadata such as scale and zero point stored in FP16. The effective number of bits per weight is

$$b_{eff} = \frac{128 \cdot 4 + 16 + 16}{128} = \frac{544}{128} = 4.25.$$

Thus, even though the raw weight mantissa uses four bits, metadata increases the actual storage cost to 4.25 bits per weight.

Quantized Linear Layer

A dense linear layer computes

$$Y = XW^T + b,$$

where $X \in \mathbb{R}^{B \times K}$, $W \in \mathbb{R}^{N \times K}$, and $Y \in \mathbb{R}^{B \times N}$. For INT4 weight-only quantization, the approximate dequantization model is

$$W_{g,j} \approx s_g (q_{g,j} - z_g),$$

where $q_{g,j}$ is a four-bit integer, s_g is the scale for group g , and z_g is the zero point.

The compute becomes

$$Y_{b,n} = \sum_{k=1}^K X_{b,k} s_{g(k,n)} (q_{n,k} - z_{g(k,n)}) + b_n.$$

A high-performance kernel must load packed weights, unpack or reinterpret them efficiently, apply scales and zero points, and feed tensor-core MMA instructions without excessive layout conversion overhead.

FlashInfer Optimization and Integration

What FlashInfer Provides

FlashInfer is described as a high-performance GPU-kernel library for LLM inference. It provides kernels for:

- attention;
- sampling;
- normalization;
- activation functions;
- paged key-value cache attention paths;
- customizable attention variants.

SGLang uses FlashInfer as a default backend. The lecture highlights close collaboration between the SGLang and FlashInfer teams, allowing early validation of upcoming FlashInfer features inside SGLang.

Customizable CUDA Templates

The upcoming FlashInfer direction described in the lecture includes a customizable CUDA template and just-in-time compilation approach inspired by flexible attention APIs. The idea is to specify an attention variant and generate optimized code for that variant.

A conceptual interface is:

```
attention_spec = {
    "mask": "causal",
    "kv_cache": "paged",
    "position_encoding": "rope",
    "dtype": "fp16",
    "backend": "cuda"
}

kernel = compile_attention_kernel(attention_spec)
out = kernel(q, paged_k_cache, paged_v_cache, metadata)
```

The important systems idea is that attention variants are numerous. Hard-coding every variant by hand is difficult. Template generation allows specialization while preserving usability.

Paged Key–Value Cache Support for FlashAttention-Style Kernels

The lecture states that standard FlashAttention-3 style kernels do not directly support paged key–value caches in the required serving form. FlashInfer adds support for paged key–value cache, which is crucial for online LLM serving.

The attention operation for one query token can be written as

$$o = \text{softmax} \left(\frac{qK^T}{\sqrt{d_h}} + m \right) V,$$

where m is the mask. With a paged cache, K and V are not necessarily contiguous in logical sequence order. The kernel must map logical token positions to physical cache pages:

$$\text{physical address}(i) = \text{page table} \left(\left\lfloor \frac{i}{P} \right\rfloor \right) + (i \bmod P).$$

This address mapping must be efficient, otherwise metadata access and pointer chasing can dominate.

Stream-K-Like Scheduling for Variable Lengths

The lecture discusses a Stream-K-inspired scheduling technique to reduce SM idle time when input lengths vary. Suppose the batch has sequence lengths

$$L_1, L_2, \dots, L_B.$$

A naive mapping may assign work per sequence, causing long sequences to occupy some SMs while others finish early. The total attention work is closer to

$$W_{total} = \sum_{i=1}^B L_i.$$

A better schedule distributes chunks of this total work across CTAs so that the runtime depends more on W_{total} and less on the maximum sequence length or variance.

If C is the number of chunks, the scheduler tries to map chunks to CTAs:

$$\phi : \{1, 2, \dots, C\} \rightarrow \{\text{CTA}_1, \text{CTA}_2, \dots\}.$$

It then needs a reduction or output mapping between partial outputs and final outputs.

Visual Concept

Draw two rows of work assignment. In the naive row, show one CTA per sequence with different bar lengths and idle gaps. In the Stream-K row, split all sequence work into equal-sized chunks and distribute them evenly across SMs.

TurboMind GEMM Optimization

Why GEMM Matters in LLM Inference

Transformer inference contains many linear layers:

$$\begin{aligned}Q &= XW_Q, & K &= XW_K, & V &= XW_V, \\O &= \text{Attention}(Q, K, V)W_O, \\ \text{MLP}(X) &= W_2 \sigma(W_1 X).\end{aligned}$$

Even in decode, GEMM and GEMV-like operations remain important. Quantized weights reduce memory bandwidth, which can be especially helpful when batch sizes are small and weight loading dominates.

Tensor Core Data Path

A tensor-core GEMM follows a staged pipeline:

1. Load operands from global memory into shared memory.
2. Load matrix fragments from shared memory into registers.
3. Feed fragments into MMA instructions.
4. Accumulate results in registers.
5. Store output back to global memory.

The simplified data path is

HBM $\rightarrow$ shared memory $\rightarrow$ register fragments $\rightarrow$ MMA $\rightarrow$ accumulators.

For standard FP16 GEMM, hardware-supported load instructions such as `ldmatrix` arrange data into register fragments expected by MMA. For low-bit operands, the packed layout may not naturally match the fragment layout expected by tensor cores.

The Low-Bit Layout Problem

Consider an MMA instruction designed around a specific fragment layout. For FP16, each element is 16 bits. For INT8 or INT4, multiple elements fit inside the storage footprint of one FP16 element. If packed data is loaded as though it were FP16, thread-to-element mapping changes.

The result is a layout mismatch:

packed low-bit memory layout $\neq$ MMA fragment layout.

Many implementations solve this by designing custom weight layouts for each architecture and each MMA instruction. The lecture argues that this is difficult to maintain because layouts differ across architectures such as Volta, Ampere, Hopper, and future GPUs.

TurboMind’s Packing Strategy

TurboMind’s idea is to use an offline packing transformation so that, at runtime, low-bit weights can be loaded through a pipeline that is compatible with standard tensor-core fragment expectations. Instead of designing many architecture-specific online layouts, it packs weights into a representation that maps directly to the required instruction behavior.

The conceptual two-stage process is:

1. **Offline packing:** transform the original low-bit weight matrix into a packed representation that matches the MMA data path.
2. **Online mixed-precision GEMM:** load packed weights efficiently, apply scale and zero-point metadata, and execute tensor-core MMA.

```
def offline_pack_int4_weights(W_int4, tile_m=16, tile_n=16):  
    # W_int4 is a logical low-bit weight matrix.  
    # The packer rearranges elements into the order expected  
    # by the runtime tensor-core GEMM pipeline.  
    packed_tiles = []  
    for tile in logical_tiles(W_int4, tile_m, tile_n):  
        expanded = bit_extend_to_16_bit_lanes(tile)  
        reordered = reorder_as_mma_fragments(expanded)  
        packed = pack_back_to_int4(reordered)  
        packed_tiles.append(packed)  
    return concatenate_tiles(packed_tiles)
```

This code is conceptual, not the exact implementation. The key idea is that expensive layout reasoning is moved to an offline packing stage so the online GEMM can be fast.

Example Tile Mapping

The lecture gives an example where a 32×32 matrix is viewed as multiple 16×16 slices. These slices are then packed into a continuous tile with shape related to 32×8 , because INT4 packing stores multiple low-bit values inside a compact word.

A simplified coordinate mapping from original matrix coordinate (m, n) to packed tile coordinates can be described as

$$t_m = \left\lfloor \frac{m}{16} \right\rfloor, \quad t_n = 16n + (m \bmod 16).$$

The exact implementation depends on the MMA instruction and architecture, but this formula captures the idea: split rows into groups of 16, then fold the row-within-tile dimension into the packed column dimension.

Advantages of the Packing Approach

The lecture emphasizes several advantages:

- It maps layouts directly to tensor-core instruction requirements.
- It is more general across GPU architectures.
- It can be loaded natively without excessive swizzling.
- It aims to avoid shared-memory bank conflicts.
- It can support power-of-two bit widths.
- The number of fragments packed together can be configurable.

This matters for serving engines because low-bit inference should not require an unmaintainable collection of separate kernels for every GPU generation, bit width, and quantization scheme.

Code Walkthrough

Scheduler Dependency with Stop Tokens

The following pseudocode shows why stop-token checking creates a dependency.

```
def process_decode_step(batch):
    tokens = run_model(batch) # Produces one token per request.

    next_batch = []
    for request, token in zip(batch.requests, tokens):
        if token == request.eos_id:
            finish_request(request)
        elif request.length + 1 >= request.max_length:
            finish_request(request)
        else:
            request.append(token)
            next_batch.append(request)

    return next_batch
```

The next batch depends on token values. In a fully blocking scheduler, this is simple. In an overlapped scheduler, token values may still be inside an unfinished GPU computation. SGLang's delayed-check strategy allows the scheduler to continue by temporarily treating the request as active.

Future Token Shape Access

A future token object allows shape-dependent scheduling code to proceed.

```

def schedule_with_future_tokens(batch, future_tokens):
    # Shape is known immediately.
    batch_size = future_tokens.shape[0]

    # Shape-dependent memory preparation can proceed.
    reserve_output_slots(batch_size)
    prepare_sampling_metadata(batch)

    # Value-dependent logic waits only when necessary.
    tokens = future_tokens.get()
    apply_stop_criteria(batch, tokens)

```

The hardware behavior is that `future_tokens.get()` becomes the synchronization point. The goal is to move this point as late as possible.

Paged Key–Value Cache Addressing

A paged key–value cache lookup can be represented as:

```

def kv_address(page_table, token_index, page_size, stride):
    page_id = token_index // page_size
    offset = token_index % page_size
    physical_page = page_table[page_id]
    return physical_page * stride + offset

```

In a CUDA kernel, this mapping must be cheap and coalesced where possible. If neighboring threads read nearby token positions, the page table and cache layout should allow efficient memory transactions. Bad layouts can cause scattered reads, poor L2 locality, and reduced bandwidth utilization.

Quantized GEMM Skeleton

A simplified mixed-precision GEMM kernel has the following logical structure.

```

// Conceptual CUDA-like pseudocode, not a complete kernel.
for each output_tile assigned to this CTA:
    load X tile from global memory to shared memory
    load packed INT4 W tile from global memory to shared memory

    for k_tile in K dimension:
        unpack or reinterpret INT4 fragments
        load activation fragments into registers
        load weight fragments into registers
        apply scale and zero-point correction
        mma(accumulator, activation_fragment, weight_fragment)

    store accumulator tile to global memory

```


The performance-critical pieces are:

- global memory loads must be coalesced;
- shared memory layout should avoid bank conflicts;
- unpacking must not dominate MMA time;
- scale and zero-point metadata should be reused across the group;
- register pressure should not reduce occupancy too far;
- tensor-core instructions should remain fed with data.

Performance Intuition

When CPU Overlap Helps Most

CPU overlap helps most when:

- decode batches are small enough that GPU work per step is short;
- request arrival and completion rates are high;
- prefix matching and memory allocation are nontrivial;
- Python scheduling overhead is visible;
- low-latency serving constraints prevent simply increasing batch size.

It helps less when GPU kernels dominate the step time so strongly that CPU time is already hidden or negligible.

Why Prefix Caching Improves TTFT

If two prompts share a prefix of length P , then without prefix caching both prompts compute the prefix separately. With prefix caching, the second request can reuse key-value states for those P tokens.

The approximate saved prefill work is proportional to

$$\Delta W_{prefill} \propto P \cdot N_{layers} \cdot d_{model}^2$$

for linear layers, plus the corresponding attention work. More importantly for serving, it saves prompt-processing time before the first generated token, so TTFT improves.

Why Variable-Length Attention Needs Better Scheduling

If sequence lengths range from 500 to 2000, naive per-sequence scheduling can leave some SMs idle after short sequences finish. Better scheduling treats the

batch as a total work pool:

$$W_{total} = \sum_i L_i.$$

This is similar to load balancing in parallel systems: runtime should depend on total work divided by available parallel resources, not on the worst imbalance induced by a poor mapping.

Why Low-Bit GEMM Can Beat FP16 GEMM at Small Batch Sizes

For small batch sizes, weight loading can dominate. INT4 weights reduce weight bytes by roughly

$$\frac{16}{4.25} \approx 3.76$$

relative to FP16 weights when the effective bit cost is 4.25 bits. This can produce large speedups if the kernel remains tensor-core efficient and metadata overhead is controlled.

At larger batch sizes, FP16 GEMM implementations such as cuBLAS become extremely efficient because they reuse weights across many batch elements and saturate tensor cores. Low-bit kernels must then compete with highly optimized dense GEMM, so speedup may shrink or even reverse if unpacking, scaling, or layout overheads become significant.

Common Misconceptions

Misconception: Fast Kernels Alone Make a Fast Inference Engine

Fast kernels are necessary but not sufficient. A serving engine also needs efficient scheduling, batching, memory allocation, prefix reuse, output processing, and network or frontend handling. CPU-side overhead can create GPU idle bubbles even when CUDA kernels are optimized.

Misconception: CPU Overhead Means the CPU Is Idle

In this lecture, CPU overhead primarily means host-side scheduling time that prevents the GPU from receiving work. The problem is not merely that the CPU waits. The problem is that the GPU, the expensive accelerator, waits while the CPU performs scheduling and bookkeeping.

Misconception: Token-Level Cache Blocks Are Always Worse for Performance

Fine-grained cache units can improve prefix sharing and reduce recomputation. Larger blocks may improve locality and simpler memory management, but they can waste reusable prefix opportunities. The best choice depends on workload structure, prompt sharing, batch size, and cache-management overhead.

Misconception: INT4 Means Exactly Four Bits Per Weight in Practice

Quantized formats store metadata. With group size 128, INT4 weights plus FP16 scale and zero point require

$$4.25$$

effective bits per weight, not exactly 4. This metadata matters for bandwidth modeling and kernel design.

Misconception: Python Threads Automatically Solve CPU Scheduling Parallelism

Python threads can help with I/O or native-code sections, but CPU-bound Python bytecode is constrained by the GIL in standard Python versions. True parallel host scheduling may require multiprocessing, native extensions, Rust components, or Python runtimes without the GIL.

Practical Takeaways

- Treat LLM serving as a pipeline with CPU and GPU stages. Look for bubbles where the GPU waits for scheduling.
- Use nonblocking GPU execution and futures to move synchronization points as late as possible.
- Delaying stop-condition checks can be a reasonable systems trade-off if it removes GPU idle time and only wastes a small amount of extra decode work.
- Prefix caching is most valuable when prompts share structure. Token-level or fine-grained cache reuse can reduce recomputation.
- Paged key-value cache support is essential for production serving because requests have dynamic lengths and memory must be reused efficiently.
- Variable-length attention needs load balancing. Kernels should distribute work according to total sequence length, not naive request boundaries.

- Low-bit GEMM performance depends on layout, packing, metadata handling, tensor-core utilization, and register pressure.
- A maintainable quantized GEMM backend should avoid one-off architecture-specific layouts when possible.

Project or Experiment Ideas

1. Implement a toy blocking scheduler and an overlapped scheduler using CUDA streams or PyTorch CUDA events. Measure GPU idle time with Nsight Systems.
2. Simulate continuous batching with random request arrivals and random output lengths. Compare delayed stop checking against exact stop checking.
3. Build a small prefix-cache simulator with block sizes 1, 8, 16, and 32. Measure prefix reuse and wasted cache space under shared-prompt workloads.
4. Benchmark attention with fixed sequence lengths versus random lengths from 500 to 2000. Inspect whether SM utilization drops for naive scheduling.
5. Implement group-wise INT4 quantization in PyTorch and compute effective bits per weight for different group sizes.
6. Compare FP16 linear layers against weight-only INT4 linear layers for batch sizes 1, 8, 32, 128, and 256. Plot latency and effective bandwidth.
7. Use Nsight Compute to inspect shared-memory bank conflicts, memory throughput, tensor-core utilization, and achieved occupancy for a custom GEMM kernel.
8. Read SGLang scheduler code and identify where request admission, prefix matching, memory allocation, and stop checking occur.

Final Mental Model

SGLang should be understood as a full-stack LLM serving engine where performance comes from coordinating multiple layers of the system. At the top, the scheduler continuously batches requests, reuses prefixes, handles stopping logic, and keeps the GPU fed. In the middle, FlashInfer provides specialized attention and inference kernels that understand paged key-value cache and variable sequence lengths. At the bottom, TurboMind-style GEMM optimization attacks the linear algebra bottleneck by making low-bit weights compatible with tensor-core execution through careful packing.

The unifying principle is simple:

Do not let expensive hardware wait, do not move bytes unnecessarily,
and do not recompute states that can be reused.

When the CPU scheduler is overlapped with GPU execution, pipeline bubbles shrink. When prefix and radix-style cache reuse work well, prefill recomputation shrinks. When FlashInfer schedules attention work well, SM idle time shrinks. When TurboMind packs low-bit weights well, memory bandwidth pressure shrinks. SGLang’s performance story is the combination of these reductions across the whole inference stack.